

Scoping of Proposed Dash Projects (July 2026, v3)

This document sizes up seven candidate Dash projects. They come from two community proposals (pay a username who is not yet a contact, make InstantSend work on transactions that have no inputs) and one privacy sketch (never reuse a UTXO), all aimed at four roadblocks, pooled masternodes, speed of capital movement to and from Platform, DashPay usability, and privacy. The projects span all three layers, Core, the mobile wallets, and the Evolution Platform.

Each section opens with a plain-English summary and then gets into the technical detail. The revision history and credits are at the end.

Summary

#	Project	Layer	Change class	Size	Verdict
P1	Instant acceptance for asset unlock transactions	Core	Spec audit, then policy (A) or quorum messaging (B)	D0 1-2 wk; A: S-M; B: L	Do first, phase-gated
P2	Coinbase maturity relaxation under ChainLocks	Core	Consensus hard fork	S code, L process	Fold into a future hard fork, not standalone
P3	Silent-payment-style static addresses	Core + Mobile	Protocol adaptation + wallet + index infrastructure	L (adapted variant), XL (Taproot path)	Strategic, research gate first
P4	No-input-merging coin selection	Mobile	Wallet policy only	S + measurement	Prototype, contact payments only
P5	Cross-chain shielded consolidation (via ZEC)	Mobile + external	Product integration	L	Drop
P6	Pay to a username that is not yet a contact	Platform + Mobile	Data contract + wallet	M-L	Park until Platform stabilizes, design now
P7	Shielded instant credit withdrawals	Core + Platform	Research-grade	XL	Decompose; long-term home of the consolidation valve

P1. Instant acceptance for asset unlock transactions (Core)

In short. When money leaves Platform and comes back to the main Dash chain, the receiving exchange or merchant waits about 2.5 minutes for a mined, ChainLocked block. That wait is a serious drag on moving capital off Platform. The fix may be cheap. These withdrawal transactions are already signed by a quorum, so the missing piece is not another signature. The missing piece is a guarantee about what happens if the transaction expires before it is mined. Pin that guarantee down, and wallets can safely credit the money the moment it appears in the mempool instead of waiting for a block.

Details. A withdrawal from Platform arrives on the Core chain as an asset unlock transaction (DIP-27, special transaction type 9). These transactions have no inputs. Each one carries a withdrawal index that must be unique, an explicit miner fee, and a signature from a Platform validator quorum (LLMQ_100_67 on mainnet). The rules refuse them once the chain grows 48 blocks past their signing height, and today they are not eligible for InstantSend. Finality comes only from being mined into a ChainLocked block.

Because the transaction is already quorum-signed when it arrives, authorization is not the gap. The gap is expiry. DIP-27 says an expired withdrawal “can be retried”, but it does not promise that the retried transaction carries the same payout script and amount. Until that promise exists, a wallet that credits a withdrawal from the mempool cannot be sure the version that finally confirms is the one it saw.

The project starts with a short audit, phase D0, one to two weeks. Read the spec and the Platform implementation and answer one question. Does a withdrawal index always bind to exactly one payout, same script and same amount, across retries? The audit also covers how unlock transactions get their fees assigned and when the mempool evicts them.

- Path A, the answer is yes, or a small spec clarification can make it yes. Then instant acceptance is just receiving policy. Wallets and exchanges credit a valid quorum-signed unlock transaction when it reaches the mempool, and published integrator guidance spreads the practice. No new network messages. The remaining risks, the transaction never getting mined or being evicted, are bounded by the retry mechanism once the payout guarantee holds. Size, small to medium.
- Path B, the answer is no, or attested finality is wanted for light clients and for chaining child transactions. Then a new quorum lock message binds the index, the txid, and the payout together. That design has real work in it. The lock must expire with the transaction it covers, unlike regular InstantSend locks, which never expire. It needs defined behavior when a conflicting unlock is mined into a ChainLocked block. Child transactions spending a locked unlock’s outputs need an explicit eligibility rule change before they can receive ordinary InstantSend locks. Quorum signing load, denial-of-service limits, persistence, the P2P and RPC surface, and reorg rules for the new conflict domain all need design, and the delivery cost includes tests, deployment, integrator rollout, and the DIP process. Size, large.

Either path attacks the capital-movement roadblock directly, D0 is cheap, and Path A may deliver most of the value at a fraction of the cost. That is why this is the first project.

P2. Coinbase maturity relaxation under ChainLocks (Core)

In short. Newly mined coins cannot be spent for 100 blocks, roughly four hours. The rule exists because a chain reorganization could erase a mined block and its reward. ChainLocks already prevent those reorganizations, so the wait is arguably obsolete. Removing it is easy code but a hard fork, so it should ride along with the next hard fork rather than justify one by itself.

Details. InstantSend is the wrong tool here, since a coinbase transaction only exists once it is mined. The barrier is the maturity rule itself, and a rule like “coinbase outputs become spendable once their block is ChainLocked” is coherent. The beneficiaries are broader than miners alone. Every block’s coinbase pays the scheduled masternode alongside the miner under consensus enforcement, and treasury proposals are paid from superbloc coinbases, so all of them wait out the same 100 blocks today. One hard design problem remains. ChainLock signatures are not stored inside blocks, and a node syncing from scratch may not have old signatures at all, so a consensus rule that depends on ChainLock state needs a concrete answer for how a syncing node validates historical spends. Keep the change drafted, answer the sync question, and attach it to whichever hard fork ships next.

P3. Silent-payment-style static addresses (Core + Mobile)

In short. The goal is an address anyone can post publicly, on a profile or a donation page, where every incoming payment lands on a fresh on-chain address that outsiders cannot connect to the owner. No address reuse, no contact handshake, no Platform involvement. Bitcoin has a spec for this, BIP352 “silent payments”, but it cannot be copied onto Dash, because it assumes Taproot and Dash does not have Taproot. Dash needs its own variant, which makes this a research project before it is a build project.

Details. Two paths.

- Taproot path. Bring Schnorr signatures, Taproot outputs, and a new bech32m address format to Dash first, then port BIP352 as written. A multi-release consensus program. Size, extra large, and not a mid-term option.
- Adapted path. Redesign the derivation for Dash’s existing ECDSA and P2PKH machinery. The recipient publishes scan and spend public keys. The sender combines them with its own keys through a Diffie-Hellman exchange and hashes the result into a normal pay-to-public-key-hash address. Cryptographically this is the BIP47 trick without BIP47’s notification transaction, and P2PKH inputs already expose the sender public keys the derivation needs. The resulting outputs are byte-identical to every other P2PKH output, so nothing on-chain marks them as special. The costs are real. A Dash-specific spec needs its own security review, none of Bitcoin’s BIP352 tooling can be reused, and the scanning burden stays, full nodes must scan every transaction to find their payments, and mobile SPV wallets need tweak-index servers. Size, large.

Nothing gets built until a research memo pins down the adapted derivation and benchmarks the scan cost on realistic Dash transaction volume.

P4. No-input-merging coin selection (Mobile, contact payments only)

In short. When a wallet pays 11 DASH from a 10 and a 1 that sit on different addresses, it merges them into one transaction, and that merge is exactly what chain-analysis firms use to cluster addresses into wallets. The proposal, never merge inputs from different addresses. Pay the 10 and the 1 as separate transactions to two addresses the recipient controls. It is a pure wallet change with no protocol involvement, but it only works where the recipient can offer multiple addresses, which means DashPay contacts, not merchant invoices. Whether the privacy gain justifies the cost is a measurement question, so the plan is a prototype behind a toggle, with hard numbers deciding ship or kill.

Details. Splitting a payment requires multiple recipient addresses. Single-address flows, BIP21 invoices, payment processors, and exchange deposit addresses present one destination and one expected amount, and a split payment can break invoice matching and refunds. The policy therefore stays off for those flows and applies only to contact-to-contact DashPay payments, where DIP-15 key exchange already lets the sender derive as many recipient addresses as needed.

The privacy gain is real but partial. The split transactions broadcast together and land in the same wallet, so their timing and amounts still correlate, and fees roughly double per payment.

A comparison with Quai's Qi ledger, which enforces the same no-reuse model at the protocol level, suggests one cheap addition. Qi allows only sixteen fixed note denominations, which coarsens amounts enough that payment size stops being a fine-grained fingerprint, though denomination patterns, output counts, and timing still leak structure. Dash CoinJoin already defines standard denominations, and a no-merge wallet could prefer those same denominations for its outputs, aiming at privacy that does not present as a flagged feature. Whether denominated no-merge payments really become hard to tell apart from CoinJoin activity is a hypothesis for the prototype to test, since CoinJoin rounds, denomination creation, and ordinary sends have different transaction shapes.

Whether the UTXO set actually grows is also not a given, since the net effect depends on change outputs and later consolidation, which is why it is measured rather than assumed.

The prototype ships behind a toggle, contact payments only, with five measured outputs. Fee overhead per payment. Net UTXO growth under simulated adoption. Re-linkage rate from a timing-and-amount correlator standing in for a chain-analysis adversary. Distinguishability of denominated no-merge payments from CoinJoin activity. Zero regressions on invoice flows. Ship or kill on those numbers.

P5. Cross-chain shielded consolidation (drop)

In short. The idea, consolidate everyone's accumulated UTXO fragments by periodically sending them through Zcash's shielded pool and back, over a cross-chain exchange. The infrastructure mostly exists now. Maya Protocol has live DASH and ZEC pools and has shipped shielded ZEC swaps. It still fails as a privacy tool, because the exchange's own chain records every swap's addresses, amounts, and timing in the clear. The round trip moves the evidence to a third ledger and pays two rounds of swap fees for the privilege.

Details. On the facts, Maya Protocol has live ZEC and DASH pools, both with status "available" in its Midgard pool data (<https://midgard.mayachain.info/v2/pools>, queried 2026-07-08). THORChain announced a phased ZEC rollout in June 2026 that was not verifiable against

live pool data at review time. Shielded-address handling has shipped on Maya, whose observers hold Zcash viewing keys that let them decrypt and verify the shielded legs they process.

None of that rescues the scheme, because the privacy stops at the Zcash leg. Every swap is recorded on the Maya chain with addresses and amounts in clear, so a consolidation round trip publishes its intent, size, and timing on a third public ledger, and amount and timing correlation reconnect the DASH ends across it.

The economics fail independently. Each user pays swap fees and slippage twice and holds ZEC price exposure during the trip. Amounts correlate in and out unless they are standardized into denominations, at which point the design has rebuilt mixing with extra steps. Coordinated consolidation intervals are themselves a fingerprint, and they concentrate demand against limited pool liquidity. Compliance desks treat shielded-ZEC-adjacent flows the way they treat mixers, so the optics motivation fails too. The honest baseline is Dash's own CoinJoin, which ships in Dash Core with mobile-wallet support following, and any consolidation-privacy proposal has to beat CoinJoin on cost, liquidity, and compliance treatment. This one does not. Dropped. P4's measurements will show whether fragmentation is even a real problem at realistic adoption.

P6. Pay to a username that is not yet a contact (Platform + Mobile)

In short. Today a DashPay username can only be paid after a mutual contact handshake, because the handshake is how wallets exchange the keys that generate fresh addresses. Paying a stranger's username requires the recipient to publish key material in advance, and it has to be the right kind. Publishing a raw extended public key would let anyone watch every payment that account branch ever receives. The workable designs publish safer material instead, a payment code or a P3-style identifier.

Details. Usernames live in the Dash Platform Name Service, and payment derivation lives in the DashPay contract (DIP-15), where mutually accepted contact requests exchange encrypted extended public keys. A plain extended public key cannot simply be made public. It exposes every address on its derivation branch, which enables surveillance of that branch's incoming payment graph, dusting, and address probing, and it puts a scanning burden on the recipient.

Two viable designs.

- BIP47-style, implementable today. Add a public payment-code field to the DashPay profile. The sender derives a one-off P2PKH address from its identity key plus the recipient's payment code, then posts a state transition as the notification that tells the recipient which derivation to watch. Platform credit fees rate-limit notification spam. One caveat from review, the notification documents sit on Platform, and even with encrypted payloads the sender-to-recipient graph edge may be visible to Platform observers, so the document and query model need a privacy review.
- Silent-payment-style, depends on P3. Publish the P3 identifier in the profile instead. No notification is needed, and Platform serves purely as a name-to-identifier directory, so payments remain valid even when Platform is unavailable. That contains Platform-instability risk to name lookup rather than funds.

Sizing, medium for the BIP47-style variant, while the P3 variant inherits the large size of P3's adapted path. Both wait on Platform stability to implement. Writing the design document now is cheap and worthwhile.

P7. Shielded instant credit withdrawals (decompose)

In short. The end goal, withdrawals from Platform that are both instant and private, is really two projects. The instant half is P1 and can start now. The private half needs something Dash does not have yet, but it matters more than it first appears. The Platform credit pool is already, structurally, the consolidation valve that P5 went looking for on other chains. Fragments go in, and withdrawals come out as fresh transactions with no UTXO-graph link to what went in. It is not a privacy tool today, because public Platform and asset-lock records still expose the link. Making those records private is the long game.

Details. Asset lock transactions deposit fragmented UTXOs into a pooled balance, and withdrawals return as zero-input asset unlock transactions with no UTXO-graph link to the deposits. That is the same shape as Quai’s protocol-level conversion between its UTXO ledger and its account ledger (a shape analogy only, Quai’s conversions are transparent, and its consolidation privacy instead comes from protocol-incentivized cooperative reaggregation, a many-party CoinJoin-like mechanism). Today the linkage survives in public records. Platform tracks identity balances and the top-up and credit-withdrawal state transitions, and the linked Core asset lock carries the deposited amount, so amounts and timing are visible on both legs, the same weakness that sank the cross-chain route in P5.

The credit pool becomes a privacy tool only if those records are made private, and that is a multi-year research question, not a scopeable feature. The dependency order is concrete. P1 makes pool round trips fast enough to use at all. P4 and P3 carry the near-term privacy roadmap. P7 is where consolidation privacy eventually lives if the research half lands. Until then, ship P1 and revisit the shielded half only as a seriously resourced research proposal.

Recommended sequence

1. P1’s D0 audit now, one to two weeks, then Path A (policy and integrator guidance) or Path B (lock message through the DIP process) as the audit dictates.
2. P4 prototype in parallel, contact payments only, with the five measurements deciding ship or kill.
3. P3 research memo next, the adapted derivation spec plus the scan-cost benchmark, before any build commitment.
4. P6 as a design document now, built once Platform stabilizes, BIP47-style variant first unless the P3 memo lands.
5. P2 folded into the next hard fork once its sync question is answered. P5 dropped. P7 reduced to P1 plus the privacy track.

Revision history

- v2 incorporated independent adversarial reviews by multiple AI models, plus primary-source checks against DIP-27, BIP352, and live decentralized-exchange pool data (2026-07-08). The material changes, P1 restructured around the D0 audit of the retry-payout invariant, P3 re-scoped for the missing Taproot dependency, P4 restricted to contact payments, P2’s beneficiary claim corrected (masternodes every block, treasury through superblock coinbases), P5’s evidence corrected in both directions, and P6’s extended-public-key risk statement corrected.

- v2.1 corrected P5 again after a further review catch. Shielded ZEC swap support has shipped on Maya Protocol, so the claim that shielded notes cannot work in these swap architectures was withdrawn, and the drop argument now rests on the Maya-chain metadata leak and the economics.
- v2.2 added two design observations from the Quai comparison. P4 gained the denominated-payments hypothesis, and P7 gained the credit-pool consolidation-valve framing.
- v3 is a plain-language rewrite after community feedback. Every section now opens with an “In short” paragraph. The technical content is unchanged from v2.2.

Credits

The originating proposals and the four-roadblock framing are due to Joel Valenzuela (TheDesert-Lynx), whose ideas on paying usernames that are not yet contacts, extending InstantSend to zero-input transactions, and wallet-level UTXO hygiene seeded this document. The scoping, the verdicts, and any errors are the author’s.

The claims in this document were adversarially reviewed by multiple independent AI models over several rounds, and every disputed claim was adjudicated against primary sources, the dashpay/dips and bitcoin/bips repositories, Dash Core source, and live decentralized-exchange pool data.